

CREADOR IA OS

Área	Estado real	Evidencia disponible	Qué significa hoy
Código del sistema	HECHO	Repositorio privado + HEAD congelado	Existe una implementación concreta, no solo documentos.
Ruta de demo	HECHO	PREPARE / VERIFY / DEMO	Hay una secuencia definida y automatizada.
Contratos de presentación	HECHO	63/63 gates · 0 faltantes	La demo tiene criterios explícitos de aceptación.
Ejecución aislada	HECHO	103 checks PASS · 0 FAIL en fase final	Los contratos y recorrido de producto fueron ejercitados en harness aislado.
Ejecución física Windows	PENDIENTE	No se ha observado PREPARE→DEMO en el equipo objetivo	Es la frontera que impide afirmar “lo vi funcionar físicamente”.
Captura visual / video	PENDIENTE	No existe todavía evidencia visual física del HEAD actual	Debe hacerse inmediatamente después del primer DEMO SUCCESS.
Merge de PR #31	PENDIENTE	PR sigue DRAFT aunque mergeable	No se debe presentar como release final integrado aún.

Qué es CREADOR IA OS

CREADOR IA OS es un sistema software IA-nativo orientado a operar proyectos de un creador/empresa con memoria, estado, continuidad y asistencia inteligente. En la FASE 1 el objetivo práctico fue llevarlo desde una arquitectura y un conjunto de módulos a un candidato reproducible que un tercero pueda preparar, verificar y demostrar de forma controlada.

Capa	Función que existe en FASE 1	Qué aporta al producto
Identidad y sesión	Registro, login, sesión, logout, autenticación JWT/local de presentación.	Permite trabajar con un creador autenticado y no con una demo anónima.
Workspace de proyecto	Creación, configuración, lectura y listado de proyectos con estado persistente.	El sistema mantiene proyecto, fase, objetivos, prioridades, bloqueos y siguientes pasos.
Runtime	Entrada de instrucciones y procesamiento dentro del contexto del proyecto.	Convierte el sistema en una herramienta operativa, no en una página estática.
Memoria / continuidad	Checkpoints, eventos y timeline.	Permite demostrar evolución y reanudar contexto.
Inteligencia	Prioridad, riesgo, learning/confirmations y snapshot visible.	La consola muestra interpretación operativa del proyecto.
Consola web	Estado, Conversación, Inteligencia y Evolución.	Da una superficie visual que un inversor puede ver.
Persistencia	PostgreSQL local de demo + migraciones/bootstrap.	Permite demostrar estado real persistido durante la sesión.
Readiness/observabilidad	Readiness interno + GET /ready con DB/schema/AI/runtime.	Permite verificar que la aplicación está lista antes de abrir el navegador.
Empaquetado de presentación	PREPARE, VERIFY, DEMO, receipt, PREPARED seal.	Transforma el repo en una demo reproducible y auditable.

IMPORTANTE: NO ES HARDWARE

CREADOR IA OS, en esta FASE 1, es software. No existe una unidad física propia que deba “construirse” como un módulo industrial. Para transferir este sistema a un tercero hay que documentar repositorio, dependencias, configuración, base de datos, scripts, API, frontend, pruebas y procedimiento de ejecución. Si en una fase futura se empaqueta en un appliance/EDGE físico, eso será otro producto/entregable.

Antes / punto de partida	Resultado de FASE 1
Diseño/arquitectura y módulos de producto sin una prueba visual reproducible única.	Candidato de demo congelado con rama dedicada, scripts de preparación/verificación/demo y criterios de aceptación.
Dependencia de saber "cómo correr" el repo.	PREPARE automatiza preflight, npm ci, verificación, cache PostgreSQL, remote checks, receipt y sello PREPARED.
Riesgo de demo no determinista.	Proveedor IA local determinista, frontend sin recursos HTTP externos, entorno de aplicación aislado de variables heredadas.
Riesgo de drift de dependencias/tooling.	Identidad por versión + fingerprint SHA-256 de node_modules y tooling PostgreSQL.
Riesgo de mostrar estado estático.	Ruta live con instrucción real de presentación, persistencia, checkpoint, timeline e Inteligencia actualizada.
Riesgo de fallos de navegador/puertos/cierre.	Fallback rundll32→explorer→manual, PORT=0, retry PostgreSQL, cleanup de señales y locks.
Criterio de "PASS" difuso.	63 gates exactos validados por receipt.

Todo lo construido en FASE 1, organizado por subsistema

4.1 Identidad, autenticación y sesión

- Flujo de registro/login y sesión de creador.
- Sesiones activas y revocación/logout.
- Ruta de presentación con auto-login y fallback manual.
- La URL manual de recuperación preserva Inteligencia sin incrustar la contraseña en la URL.

4.2 Workspace y estado del proyecto

- Creación de proyecto y estado inicial.
- Configuración de fase, objetivos, prioridades, bloqueos, siguientes pasos y contexto.
- Reapertura/listado preservando el estado operativo.
- Seed de presentación con un proyecto visible: ADNEKO Launch System.

4.3 Runtime de interacción

- Servicio de runtime accesible por la misma ruta HTTP que usa el producto.
- Idempotencia de la instrucción de verificación.
- Respuesta visible en conversación.
- Persistencia de la interacción en el historial.

4.4 Memoria, checkpoints y continuidad

- Evento checkpoint-created.
- Checkpoint con ID verificable.
- Timeline de continuidad.
- La ruta de presentación exige un checkpoint adicional durante la interacción live (timeline >= 2).
- Reload evita duplicar la instrucción live ya ejecutada.

4.5 Inteligencia visible

- Snapshot de prioridad/riesgo/estado de inteligencia mostrado en la consola.
- Persistencia temporal del último snapshot de Inteligencia para reload accidental.
- Invalidación/recarga de Evolución después de la acción live.

4.6 Consola web

- Superficies visibles: Estado, Conversación, Inteligencia y Evolución.
- Autenticación, apertura del proyecto, envío de instrucción y actualización visual.
- Sin dependencia de CDN, Google Fonts ni recursos HTTP(S) externos en los assets auditados de presentación.

4.7 PostgreSQL y bootstrap

- PostgreSQL embebido/local para la demo.
- Cache persistente del tooling de PostgreSQL.
- Fingerprint de runtime + paquete binario de plataforma.
- Retry de colisión de bind PostgreSQL hasta 3 intentos, con puerto y data directory nuevos.

4.8 Readiness y HTTP

- Readiness interno del servicio.
- GET /ready público obligatorio antes del browser handoff.
- Checks requeridos: database, schema, ai y runtime.
- Verificación de console/assets/login/session/project/interactions/operational/pending/timeline/logout.

4.9 Seguridad/determinismo de presentación

- NODE_OPTIONS y NODE_PATH neutralizados en PREPARE/VERIFY/DEMO.
- Entorno de aplicación no hereda process.env para los controles de comportamiento.
- Pagos, cuotas IA y project limits explícitamente desactivados en preview.
- Transporte IA fail-closed a preview.invalid; no se usa proveedor externo en la demo.

4.10 Robustez del operador

- Browser fallback: rundll32 → explorer → recuperación manual.
- Puerto HTTP asignado atómicamente por el SO con PORT=0.
- SIGINT/SIGTERM durante startup ejecutan cleanup.
- Cleanup compartido e idempotente.
- Retries de eliminación de tempRoot ante locks Windows.
- PREPARE imprime NEXT: CREADOR-OS-DEMO.cmd.
- DEMO imprime SUCCESS únicamente después de exit code 0.

Archivos concretos

Archivo / grupo	Para qué sirve	Si se pierde el chat, qué conserva
package.json + package-lock.json	Dependencias y comandos de ejecución/prueba.	Reproducibilidad de dependencias y scripts.
CREADOR-OS-PREPARE.cmd	Preflight, sync remoto, npm ci, VERIFY, cache, receipt y PREPARED.	Procedimiento de preparación reproducible.
CREADOR-OS-VERIFY.cmd	Ejecuta la certificación completa y valida receipt.	Criterio técnico de PASS previo a demo.
CREADOR-OS-DEMO.cmd	Valida receipt/PREPARED y lanza el fast live path.	Procedimiento de presentación.
tools/run-presentation-demo.mjs	Runner de certificación y fast launch.	Orquestación de gates y demo.
tools/run-free-preview.mjs	PostgreSQL local, bootstrap, seed, HTTP checks, browser handoff.	Runtime local de la demostración.
tools/validate-presentation-receipt.mjs	Autoridad de los 63 gates.	Contrato exacto de certificación.
tools/presentation-prepared-evidence.mjs	Sella receipt + SHA + run id + remoto.	Prueba de que PREPARE corresponde al candidato actual.
tools/presentation-dependency-contract.mjs	Valida lock/versions/fingerprint de dependencias.	Protección contra drift de node_modules.
tools/presentation-tooling-cache.mjs	Valida/fingerprints del PostgreSQL tooling.	Protección contra drift del runtime/binario PG.
frontend/*	Consola visible del producto.	Interfaz que el inversor puede ver.
backend/*	Servicios de auth, workspace, runtime, continuidad, API y bootstrap.	Implementación real del producto.

tests/unit/presentation-*	Contratos de demo y regresiones.	Cómo evitar romper la presentación.
tests/integration/creator-product-journey.integration.test.js	Recorrido funcional de producto.	Prueba de registro→proyecto→runtime→timeline→logout.

6. Evidencia disponible

Área	Estado real	Evidencia disponible	Qué significa hoy
HEAD congelado	CONFIRMADO	307055fe47e464cc824d8a75900b4304cf2e5f31	Identidad exacta del candidato de cierre.
PR #31	CONFIRMADO	OPEN · DRAFT · MERGEABLE · [FROZEN]	El cambio es mergeable pero aún no es release integrado.
Gates	CONFIRMADO	63 requeridos · 0 faltantes	Runner/validator están alineados.
Contratos unitarios presentación	EJECUTADO	88/88 PASS · 0 FAIL	Los 12 archivos unitarios de presentación fueron ejercitados en la fase final.
Recorrido producto aislado	EJECUTADO	15/15 PASS · 0 FAIL	Auth→sesión→proyecto→runtime→checkpoint→timeline→logout.
Total fase final	EJECUTADO	103 PASS · 0 FAIL	Evidencia aislada; no equivale a PREPARE físico Windows.
PREPARE físico en Windows objetivo	NO EJECUTADO	Sin receipt físico de esa máquina	Pendiente indispensable para afirmar “lo vi funcionar”.
DEMO físico / navegador	NO EJECUTADO	Sin screenshot/video del HEAD actual	Pendiente para evidencia comercial visual.

LO QUE LOS 103 PASS NO SIGNIFICAN

No significan que el sistema ya fue visto físicamente en el Windows de presentación. Significan que los contratos de FASE 1 y el recorrido de producto fueron ejercitados en un entorno aislado y no produjeron fallos. La afirmación “funciona físicamente en esa máquina” solo debe hacerse después de PREPARE + DEMO exitosos allí.

7. Qué se debe poder ver materialmente cuando se ejecute

Momento	Qué debe aparecer	Por qué es importante
Terminal PREPARE	PREPARED · Verification PASS · Prepared seal SEALED · NEXT: CREADOR-OS-DEMO.cmd	Prueba que máquina, rama, dependencias, PostgreSQL, gates y remoto quedaron preparados.
Inicio DEMO	Verification receipt: PASS · Preparation seal: PASS	Prueba que no se está ejecutando evidencia vieja.
Navegador	Consola CREADOR IA OS / proyecto ADNEKO Launch System	Primera evidencia visual del producto.
Estado	Fase, prioridades, bloqueos y próximos pasos	Demuestra estado operativo persistente.
Conversación	Historial + instrucción live + respuesta contextual	Demuestra interacción real a través del runtime.
Inteligencia	Prioridad/riesgo/estado actualizado	Demuestra interpretación operativa, no una página estática.
Evolución	Checkpoint nuevo / timeline actualizado	Demuestra continuidad y evolución después de la acción.
Cierre	CREADOR IA OS - PRESENTATION DEMO: SUCCESS	Prueba que el runner terminó sin error.

Qué falta exactamente para verlo funcionar

EL GAP ACTUAL ES CORTO Y CONCRETO
No hace falta escribir otro sistema para poder verlo. Hace falta ejecutar el candidato exacto en una máquina Windows con acceso al repositorio, dejar que PREPARE genere su evidencia, lanzar DEMO y capturar el resultado.

Pendiente	Acción exacta	Criterio de cierre
1. Máquina objetivo	Windows con Git, Node >=20 y npm disponibles.	PREPARE preflight = PASS.
2. Checkout exacto	Rama presentation/creator-os-final actualizada al HEAD remoto congelado.	git rev-parse HEAD = 307055fe... y working tree CLEAN.
3. Preparación	Ejecutar CREADOR-OS-PREPARE.cmd.	PREPARED + Verification PASS + Prepared seal SEALED.
4. Demo	Ejecutar CREADOR-OS-DEMO.cmd.	Navegador abre y cierre con PRESENTATION DEMO: SUCCESS.
5. Evidencia visual	Tomar capturas/video de terminal + Estado + Conversación + Inteligencia + Evolución.	Paquete visual listo para inversor.
6. Si aparece un error físico	Corregir solo ese blocker y repetir PREPARE.	Nuevo receipt/PREPARED válidos y demo exit 0.

Procedimiento oficial de reconstrucción y ejecución desde cero

9.1 Prerrequisitos

- Windows 10/11 o entorno Windows equivalente para la ruta de presentación .cmd.
- Git instalado y origin configurado con acceso al repositorio privado.
- Node.js 20 o superior. La evidencia aislada de esta fase usó Node v22.16.0.
- npm disponible como npm.cmd.
- Acceso de red a GitHub durante PREPARE para resolver origin/presentation/creator-os-final e instalar dependencias si no están cacheadas.

9.2 Obtener el candidato exacto

```
git clone <REPOSITORIO_PRIVADO> creator-ai-os
cd creator-ai-os
git fetch origin presentation/creator-os-final
git checkout -B presentation/creator-os-final origin/presentation/creator-os-final
git rev-parse HEAD
# Debe corresponder al candidato congelado documentado: 307055fe47e464cc824d8a75900b4304cf2e5f31
```

9.3 Preparar

CREADOR-OS-PREPARE.cmd
PREPARE realiza automáticamente:

- Limpieza de evidencia PREPARED vieja.
- Validación Node/npm/Git.
- Validación de rama exacta y working tree CLEAN.
- Comparación local SHA vs origin/presentation/creator-os-final.
- Generación de PREPARE_RUN_ID.
- npm ci --no-audit --no-fund (dependencias exactas desde package-lock).
- Llamada a VERIFY con PostgreSQL cache warming.
- Revalidación del SHA remoto después de VERIFY.
- Creación del sello PREPARED asociado al receipt y al candidato.

9.4 Verificación automática (dentro de PREPARE)

CREADOR-OS-VERIFY.cmd # PREPARE lo ejecuta automáticamente

- Elimina receipt viejo.
- Ejecuta tools/run-presentation-demo.mjs en VERIFY_ONLY.
- Genera .presentation-evidence\creator-os-verify.json.
- Valida el receipt con tools/validate-presentation-receipt.mjs.
- Solo retorna exit code 0 si la autoridad completa es válida.

9.5 Ejecutar la demo

CREADOR-OS-DEMO.cmd

- Exige receipt y PREPARED existentes.
- Revalida ambos antes de lanzar.
- Revalida Node, dependencias, fingerprints, branch/HEAD/tree y tooling PostgreSQL.
- Arranca PostgreSQL local/efímero.
- Arranca la aplicación con PORT=0 y obtiene el puerto asignado por el SO.
- Ejecuta seed y readiness público.
- Abre el navegador mediante rundll32; si falla usa explorer; si ambos fallan mantiene servidor vivo y muestra URL manual.
- Permite ejecutar la instrucción live de continuidad y comprobar checkpoint/evolución.
- Al cerrar, debe terminar con PRESENTATION DEMO: SUCCESS.

9.6 Evidencia que debe conservarse

- .presentation-evidence\creator-os-verify.json
- .presentation-evidence\creator-os-prepared.json
- Captura de PREPARE PASS/SEALED.
- Captura de DEMO receipt/prepared PASS.
- Capturas de Estado, Conversación, Inteligencia y Evolución.
- Captura final PRESENTATION DEMO: SUCCESS.
- SHA mostrado por git rev-parse HEAD.

Si algo falla: matriz de diagnóstico

Síntoma	Qué significa	Acción correcta
PREPARE: branch FAIL	No está en presentation/creator-os-final.	Cambiar a la rama correcta; no saltarse el control.
PREPARE: working tree FAIL	Hay cambios locales.	Guardar/revisar cambios; la demo exige tree limpio.
Remote candidate FAIL/CHANGED	Local y remoto no son el mismo candidato.	Sincronizar rama y repetir PREPARE.
npm ci FAIL	Dependencias no pudieron reproducirse.	Resolver acceso/npm/lock; no lanzar DEMO con node_modules incompleto.
Receipt invalid/stale	La evidencia corresponde a otro SHA/Node/fingerprint.	Repetir PREPARE.
PREPARED invalid	El sello no corresponde al receipt/candidato actual.	Repetir PREPARE; no crear el sello manualmente.
Tooling fingerprint mismatch	Cambió el cache/binario PostgreSQL.	Repetir PREPARE para regenerar cache y evidencia.
/ready no está ready	DB/schema/AI/runtime no están listos.	Corregir el check fallido antes de abrir navegador.
Browser auto-open falla	Problema de handoff de Windows, no necesariamente de app.	Usar fallback explorer o URL manual mostrada.
Port PostgreSQL ocupado	Colisión real de bind.	El sistema reintenta hasta 3 veces; si persiste, revisar procesos locales.
DEMO: FAILED	Runner terminó con exit code != 0.	Tomar el error exacto, corregir solo ese blocker, repetir PREPARE.

Qué falta

Esta sección separa “lo necesario para verlo funcionar” de “lo necesario para declararlo producto operativo en producción”. La primera lista es corta; la segunda exige decisión de release.

Área	Estado después de DEMO SUCCESS	Trabajo de producción pendiente
Release	Demo candidate probado.	Decidir merge/release desde PR #31 y etiquetar versión.
Despliegue	Demo local reproducible.	Deployment reproducible en infraestructura objetivo, secretos y rollback.
IA externa	Demo usa transporte local determinista.	Configurar y certificar proveedor real, cuotas/costos/fallbacks si el producto los requiere.
Base de datos	PostgreSQL local funciona como demostración.	Provisionar PostgreSQL de producción, backups, recovery y observabilidad de entorno.
Seguridad	Contratos de presentación y auth existen.	Revisión de seguridad del deployment real, gestión de secretos, TLS y políticas operativas.
Operación	Readiness/health y logs existen a nivel de sistema.	Monitorización, alertas, runbook y soporte operativo del entorno final.
Comercial	Producto demostrable.	Definir packaging/licencia/precio/SLA/onboarding y material comercial final.

Guion de preguntas

Pregunta	Respuesta precisa
¿Qué es?	Un sistema operativo software IA-nativo para gestionar proyectos de un creador/empresa con estado, conversación, inteligencia y continuidad persistente.
¿Existe o es un concepto?	Existe en un repositorio privado con backend, frontend, scripts de ejecución, base de datos local de demo y contratos de prueba. No es solo un documento.
¿Qué se puede ver?	Una consola con Estado, Conversación, Inteligencia y Evolución, más un flujo live que crea un checkpoint y actualiza el timeline.
¿Está terminado?	FASE 1 de ingeniería/demostración está cerrada. La ejecución física final en el Windows objetivo sigue pendiente. Producción v1.0 requiere release/deployment posteriores.
¿Qué pruebas tienen?	63 gates definidos, 0 faltantes; 88/88 contratos unitarios de presentación y 15/15 checks de recorrido de producto aislado, total 103 PASS / 0 FAIL en la fase final.
¿Por qué no hay screenshot todavía?	Porque no se ha ejecutado físicamente el HEAD final en el Windows de presentación. El paquete para hacerlo ya existe.
¿Qué falta para enseñármelo?	Ejecutar CREADOR-OS-PREPARE.cmd y luego CREADOR-OS-DEMO.cmd en el equipo objetivo y capturar el resultado.
¿Depende de Internet para la demo?	La ruta de IA y los assets de la consola están diseñados para ser offline/deterministas durante la presentación. PREPARE sí necesita acceso al repositorio/npm cuando corresponde.
¿Es hardware?	No en esta fase. Es software. Un appliance físico sería otro producto/fase.

Tablero final de estado FASE 1

Área	Estado real	Evidencia disponible	Qué significa hoy
Arquitectura/producto	CERRADO FASE 1	Servicios y rutas concretas en repo	No abrir rediseño para lograr la demo.
Demo reproducible	CERRADO EN CÓDIGO	PREPARE/VERIFY/DEMO + receipt/PREPARED	Ruta definida y fail-closed.
Pruebas aisladas	PASS	103 PASS · 0 FAIL	Da evidencia técnica previa al equipo físico.
Windows físico	ABIERTO	No ejecutado en equipo objetivo	Único blocker para afirmar prueba visual física.
Evidencia visual	ABIERTO	Sin screenshots/video del HEAD final	Capturar tras primer SUCCESS.
Release/merge	ABIERTO	PR #31 DRAFT	Decisión posterior a demo física.
Producción v1.0	FASE SIGUIENTE	No declarada en este documento	Deployment/release/ops/comercial.

APÉNDICE A · Los 63 gates de certificación de presentación

#	Gate	Esperado
1	branchIdentity	PASS
2	workingTree	CLEAN
3	finalWorkingTree	CLEAN
4	repositoryDependencies	PASS
5	repositoryDependencyIdentity	PASS
6	repositoryDependencyContentIdentity	PASS
7	dependencyExecutionAfterIdentityContract	PASS
8	syntax	PASS
9	presentationContract	PASS
10	commandModelsolation	PASS
11	machinePreflightContract	PASS
12	runtimeNodeIdentity	PASS
13	nodeEnvironmentIsolationContract	PASS
14	directNodeExecutionContract	PASS
15	certifiedFastLaunchContract	PASS
16	finalLiveGitIdentityContract	PASS
17	nonInteractivePrepareContract	PASS
18	remoteCandidateSyncContract	PASS
19	toolingCacheIdentity	PASS
20	toolingContentIdentity	PASS
21	presentationSeedFlow	PASS
22	httpLiveRuntime	PASS
23	offlineAITransportContract	PASS
24	applicationEnvironmentIsolationContract	PASS
25	liveInstructionPersistence	PASS
26	liveEvolutionFlow	PASS
27	liveIntelligenceReloadContract	PASS
28	browserHandoffContract	PASS
29	browserFallbackContract	PASS
30	browserManualRecoveryContract	PASS
31	browserManualIntelligenceContract	PASS
32	offlineBrowserAssetsContract	PASS
33	cleanShutdownContract	PASS
34	startupSignalCleanupContract	PASS
35	tempCleanupResilienceContract	PASS
36	distinctPortAllocationContract	PASS
37	databasePortCollisionRetryContract	PASS
38	receiptContractValidation	PASS
39	preparationAuthorityContract	PASS
40	prepareBoundReceiptContract	PASS
41	creatorJourney	PASS
42	postgresBootstrap	PASS
43	presentationSeed	PASS
44	runtimeInteraction	PASS
45	runtimeCheckpointEvent	PASS
46	noPendingConfirmation	PASS

47	continuityCheckpoint	PASS
48	presentationIntelligence	PASS
49	serviceReadiness	PASS
50	httpReadiness	PASS
51	httpConsole	PASS
52	httpAssets	PASS
53	httpLogin	PASS
54	httpSession	PASS
55	httpProject	PASS
56	httpInteractions	PASS
57	visibleConversation	PASS
58	httpOperational	PASS
59	operationalDashboard	PASS
60	httpPendingActions	PASS
61	pendingConfirmation	NONE
62	httpTimeline	PASS
63	httpLogout	PASS

APÉNDICE B · Checklist de entrega a un tercero

- ☐ Acceso al repositorio privado concedido.
- ☐ Rama presentation/creator-os-final localizada.
- ☐ HEAD verificado contra el candidato documentado.
- ☐ Node/npm/Git disponibles.
- ☐ Working tree limpio.
- ☐ PREPARE ejecutado y evidencia guardada.
- ☐ PREPARED seal validado.
- ☐ DEMO ejecutado hasta SUCCESS.
- ☐ Capturas de las cuatro superficies guardadas.
- ☐ Receipt + PREPARED + SHA archivados con la entrega.
- ☐ PR/release decision registrada.
- ☐ Runbook de producción abierto como siguiente fase.

APÉNDICE C · Resumen de autoridad del documento

Este documento consolida el estado técnico conocido del repositorio CREADOR IA OS al cierre de FASE 1 y distingue explícitamente evidencia de código/contratos de la ejecución física aún pendiente. Los números y SHA consignados corresponden al candidato de presentación congelado documentado en PR #31.

Regla de mantenimiento: si el HEAD cambia después de este corte, se debe actualizar este documento o generar una nueva revisión, porque receipt/PREPARED y fingerprints están ligados a la identidad exacta del candidato.